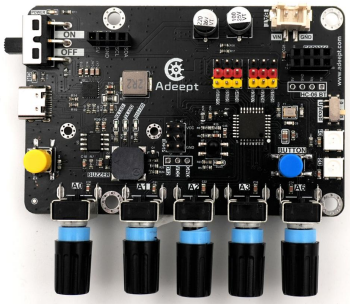
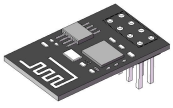
Lesson 11 How to Perform Wireless Communication with ESP8266

11.1 Overview

In this lesson, we will introduce the ESP8266 module and demonstrate how to realize wireless communication with it through an example.

Note: The ESP8266 draws relatively large operating current. Be sure to power it with a battery. Powering solely via USB may affect communication functions.

11.2 Required Components

Components	Quantity	Picture
Adeept Robotic Arm Control Board	1	
Type-C USB Cable	1	
ESP8266	1	

OLED Screen	1	

11.3 Principle Introduction

- Core Positioning and Basic Working Principle of ESP8266 Module

The ESP8266 is a low-cost, highly integrated 2.4G Wi-Fi System-on-Chip (SoC) module developed by Espressif. Its core operating logic centers on wireless communication and control. It integrates Wi-Fi communication circuitry and a 32-bit microcontroller into a single unit. It can work as an auxiliary module to add wireless capability to other main controllers, or operate independently as a master controller to deliver wireless communication for IoT applications, making it one of the most widely used modules for beginners entering wireless IoT development.

In terms of core operating mechanisms, the ESP8266 embeds an RF power amplifier, filter circuits and power management modules. It transmits and receives wireless signals via its built-in Wi-Fi protocol stack. Paired with its native 32-bit microcontroller, it processes data, parses and executes instructions, enabling wireless data exchange between devices, and between devices and cloud servers. Its core functionality is reflected in three Wi-Fi operating modes and two development approaches, detailed below:

(1) Three Wi-Fi Operating Modes

The ESP8266 switches between different Wi-Fi modes to fit various wireless communication scenarios, leveraging its onboard Wi-Fi hardware to establish connections and transmit data as required:

1) STA (Station) Mode (Most Commonly Used) In this mode, the ESP8266 acts as a wireless client that searches for and connects to existing Wi-Fi networks such as home routers to access the internet. The module completes handshakes and authentication with the router through its internal Wi-Fi protocol stack to build a stable network link. Once connected, it can upload sensor data to the cloud and support remote hardware control via mobile phones or cloud platforms, such as wireless manipulation of robotic arms and adjustment of LED brightness.

2) AP (Access Point) Mode Here, the ESP8266 generates its own independent Wi-Fi hotspot without relying on a router. Mobile phones, computers and other terminals

can search and connect directly to this hotspot. The module actively broadcasts Wi-Fi signals to establish point-to-point local communication with connected devices for direct data interaction. This mode is ideal for equipment debugging and local control without internet access, such as wireless operation of robotic arms during on-site testing.

3) STA+AP Hybrid Mode This mode combines the advantages of STA and AP. The module can simultaneously connect to an external router for cloud communication and activate a local Wi-Fi hotspot for direct local device access. While maintaining internet connectivity via STA for remote data transmission, it receives commands from local terminals through the AP channel. It balances cloud remote control and on-site local debugging for greater flexibility.

(2) Two Development Approaches

Two methods are available to utilize the ESP8266' s functions, tailored to different project requirements. Wireless operations are controlled via dedicated commands or custom programs:

1) AT Command Transparent Transmission Mode (Matched with Arduino/STM32) The module comes pre-flashed with AT firmware from the factory. It communicates with external main controllers like Arduino and STM32 through serial ports. The external controller sends simple AT commands (for Wi-Fi connection, data transmission, etc.) to instruct the ESP8266 to establish Wi-Fi links and send/receive TCP/UDP data. This method eliminates the need to learn ESP-specific programming. Users can add wireless networking to existing hardware with basic commands, suitable for retrofitting legacy equipment with wireless functionality.

2) Independent MCU Programming Mode (For NodeMCU Development Boards) NodeMCU boards are built around the ESP8266 chip. Developers can write custom code using the Arduino IDE, MicroPython or official SDKs and flash it to the module. The ESP8266 then runs standalone as a master controller. Its internal 32-bit microcontroller executes custom code to handle Wi-Fi management, data processing and peripheral driving without an external main controller, ideal for building standalone IoT projects.

- Core Advantages of the ESP8266 Module
  - Ultra cost-effective: Entry-level variants such as ESP-01 are affordably priced, drastically cutting hardware costs for mass production and learning projects.

- High integration: Integrated RF power amplifier, filters and power management circuits minimize external peripheral components and simplify circuit design and assembly.
- Low-power design: Three power-saving profiles are supported: active mode, light sleep and deep sleep. Deep-sleep current is only 20μA, extending battery life for battery-powered smart home and portable devices.
- Comprehensive protocol support: Compliant with 802.11 b/g/n Wi-Fi standards, it supports TCP/UDP, HTTP, MQTT, SSL encryption and other protocols, enabling seamless compatibility with most cloud platforms including Alibaba Cloud and Tencent Cloud for diverse wireless data exchange.
- Low development barrier: One-click compatibility with Arduino plugins removes complex configuration steps, allowing new developers to quickly build Wi-Fi control projects such as wireless robotic arm operation and sensor data uploading to the cloud.

- Project Model: ESP-01S

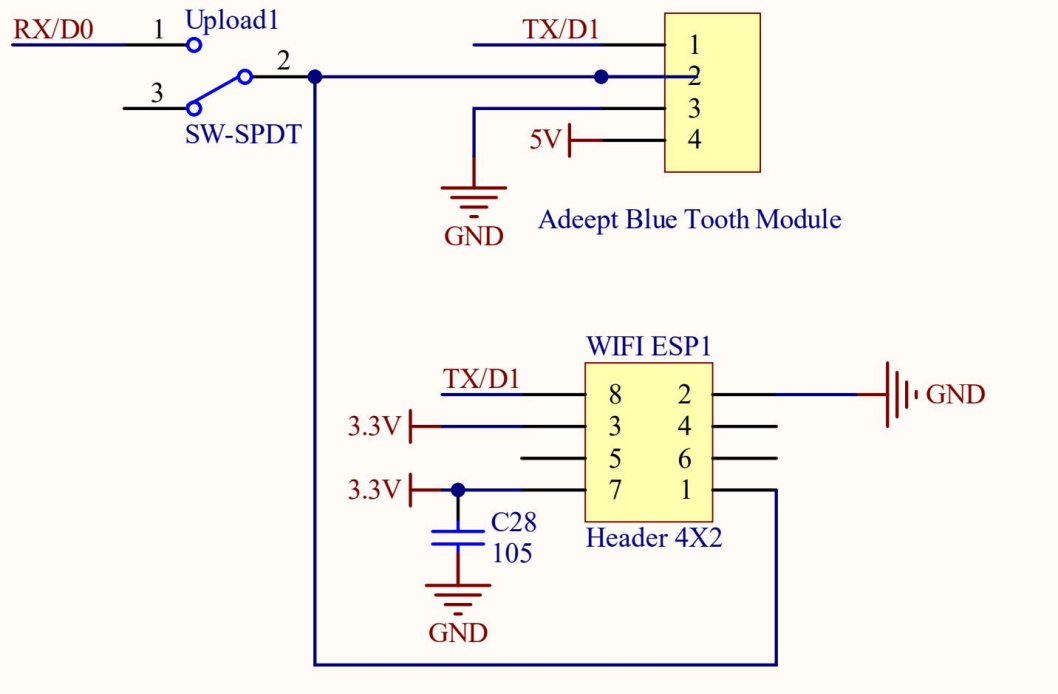
The ESP-01S used in our projects is a classic compact Wi-Fi module within the ESP8266 lineup and an upgraded version of the original ESP-01. It shares the core working principles of all ESP8266 chips while offering unique benefits optimized for our robotic arm and Arduino control board projects:

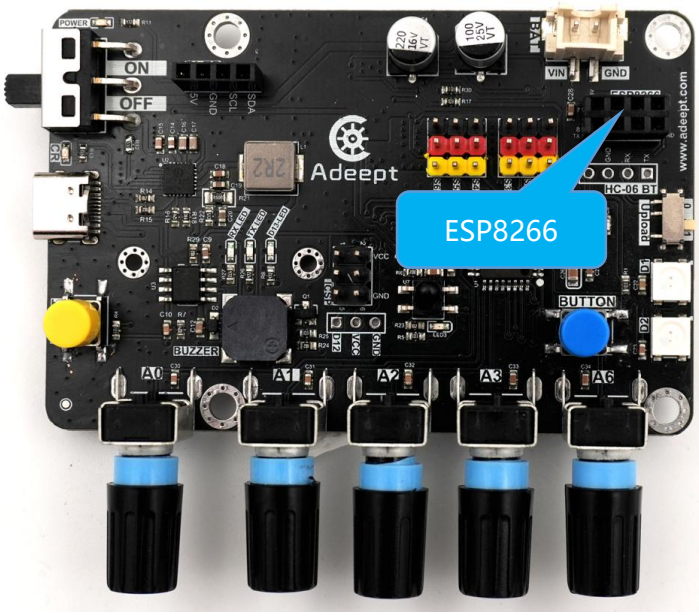
1. Hardware compatibility benefits: Equipped with 4MB Flash memory (compared to just 1MB on the older ESP-01), it stores more programs, web page files and network credentials to avoid runtime errors caused by insufficient memory. Its ultra-small form factor with only 8 pins allows easy embedding onto control boards without occupying excessive space.
2. Target project applications: Primarily used to add wireless communication capability to Arduino main controllers. Paired with Arduino via AT transparent transmission, it enables remote robotic arm control via computers or mobile phones, and wireless uploading of sensor readings including potentiometer values and battery voltage data, fully matching our project demands.
3. Enhanced operational stability: As an upgraded model, it delivers stronger signal reception and smoother performance than the original ESP-01, ensuring timely, accurate data transmission over Wi-Fi and preventing frequent disconnections or data loss.

- Typical Application Scenarios (Practical Implementation of Working Principles)

  1. Smart home: Connect to the internet via STA mode to build wireless switches, smart lighting and remote temperature/humidity monitoring systems, enabling users to control household devices remotely via mobile phones.
  2. Robotics projects: As demonstrated by our robotic arm solution, the ESP-01S adds Wi-Fi wireless capability for upper-computer control and parameter adjustment via computers or mobile phones. It is also widely deployed for remote control of smart cars.
  3. Data acquisition systems: Paired with various sensors, it wirelessly transmits temperature, humidity, voltage, light intensity and other readings to the cloud for remote monitoring and data analysis.
  4. Educational experimental platforms: Serves as a training module for single-chip microcontroller beginners to master Wi-Fi communication theory, AT command operation and serial transparent wireless transmission logic.

11.4 Wiring Diagram





PINS of Adeept Robotic Arm Control Board	ESP-01S
RX/D0	RX
TX/D1	TX
3.3V	3V3
GND	GND
3.3V	EN

Both ESP8266 communication and microcontroller program flashing occupy serial port resources. To resolve this conflict, we have specially designed an upload toggle switch.

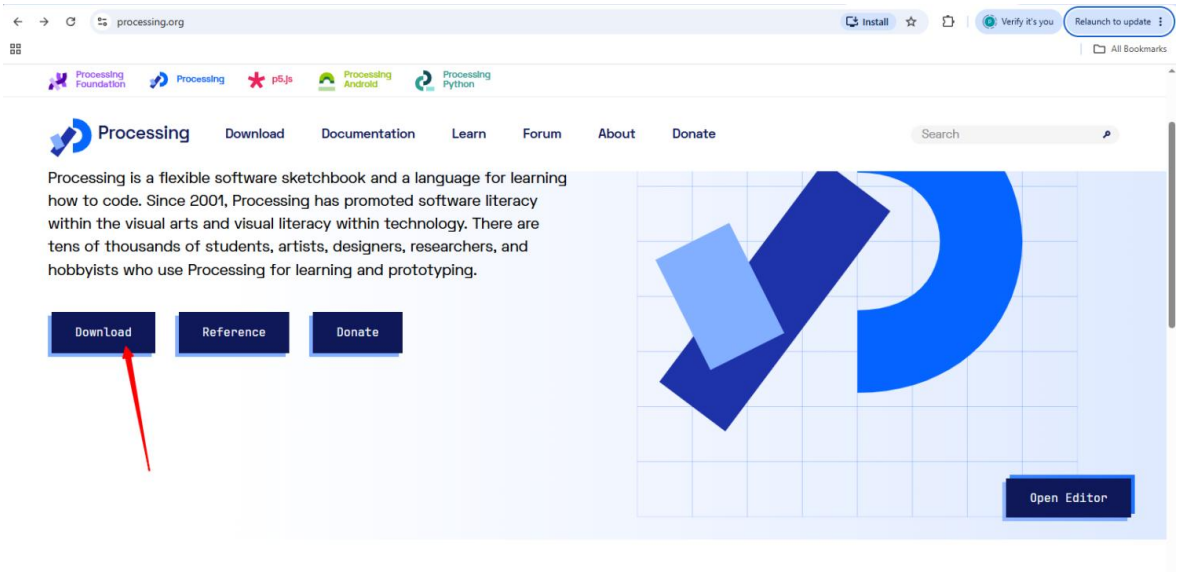
When the switch is toggled to gear **0**, it enters program upload mode for flashing firmware to the main control board.

When toggled to gear **1**, it switches to ESP8266 communication mode, under which the microcontroller can receive data transmitted from the ESP8266 client.

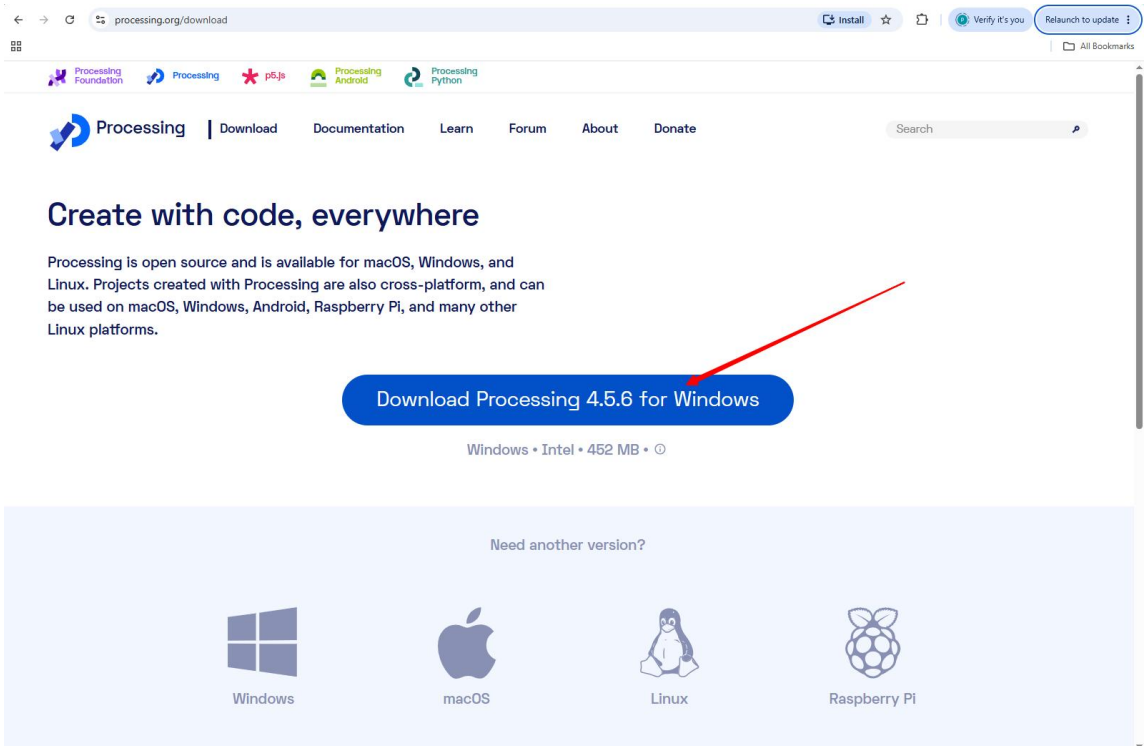
11.5 Install Processing

We used Processing in the example, so let’ s install it first.

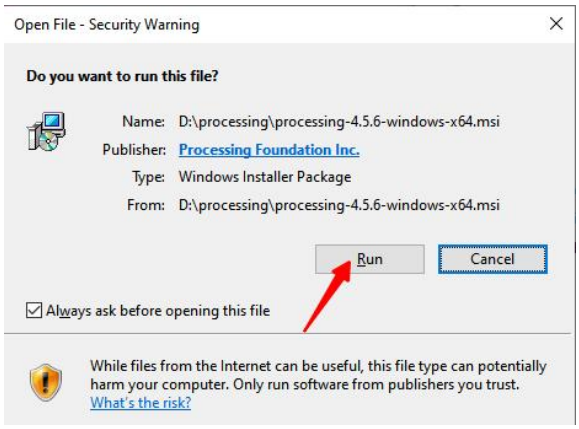
1.Open your browser and enter the official Processing website address:https://processing.org/

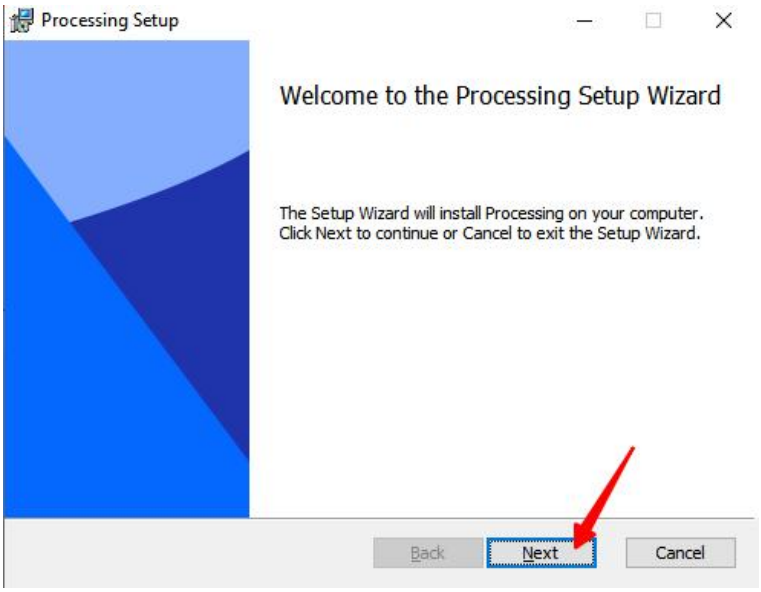


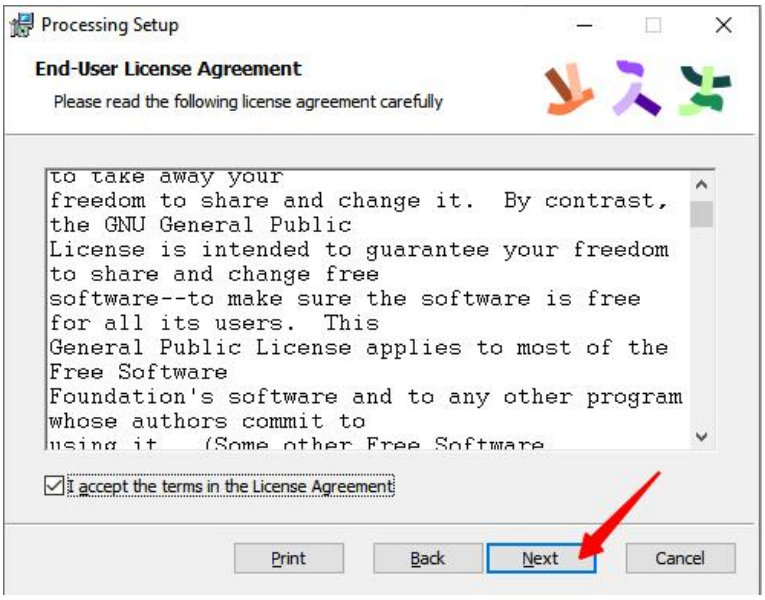
2.Click "**Download**" to download the version compatible with your computer.

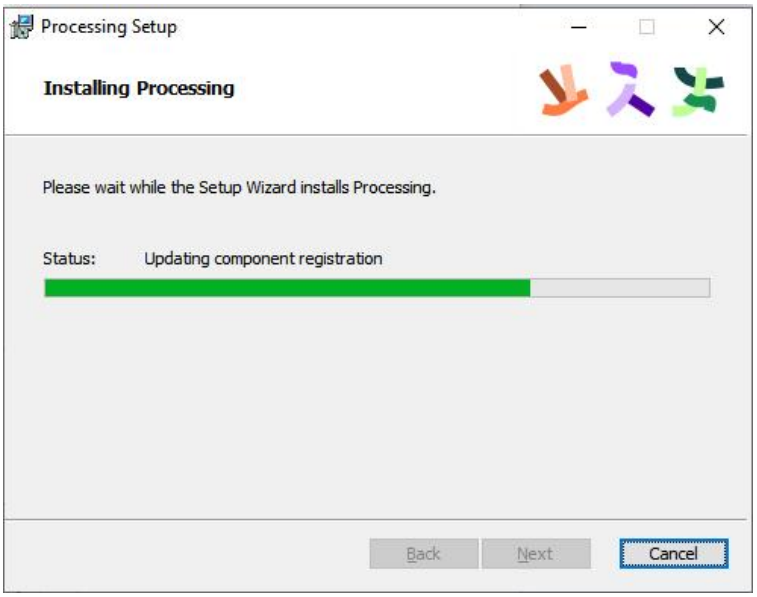


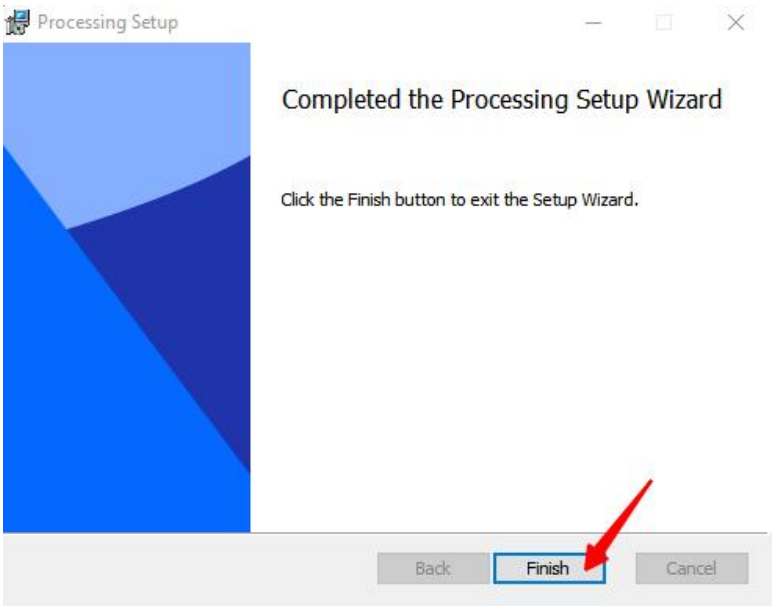
3. Wait for the download to finish, then install it following the prompts.


Open File - Security Warning
Do you want to run this file?
Name: D:\processing\processing-4.5.6-windows-x64.msi
Publisher: Processing Foundation Inc.
Type: Windows Installer Package
From: D:\processing\processing-4.5.6-windows-x64.msi
Run
Cancel
Always ask before opening this file
While files from the Internet can be useful, this file type can potentially harm your computer. Only run software from publishers you trust.
What's the risk?

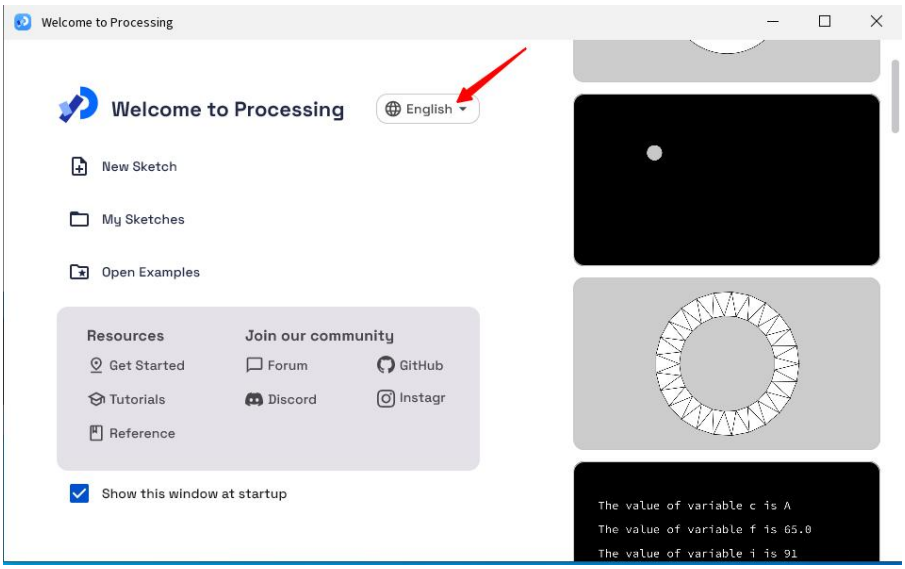

Processing Setup
Welcome to the Processing Setup Wizard
The Setup Wizard will install Processing on your computer.
Click Next to continue or Cancel to exit the Setup Wizard.
Back
Next
Cancel


Processing Setup
End-User License Agreement
Please read the following license agreement carefully
to take away your
freedom to share and change it. By contrast,
the GNU General Public
License is intended to guarantee your freedom
to share and change free
software--to make sure the software is free
for all its users. This
General Public License applies to most of the
Free Software
Foundation's software and to any other program
whose authors commit to
I accept the terms in the License Agreement
Print
Back
Next
Cancel

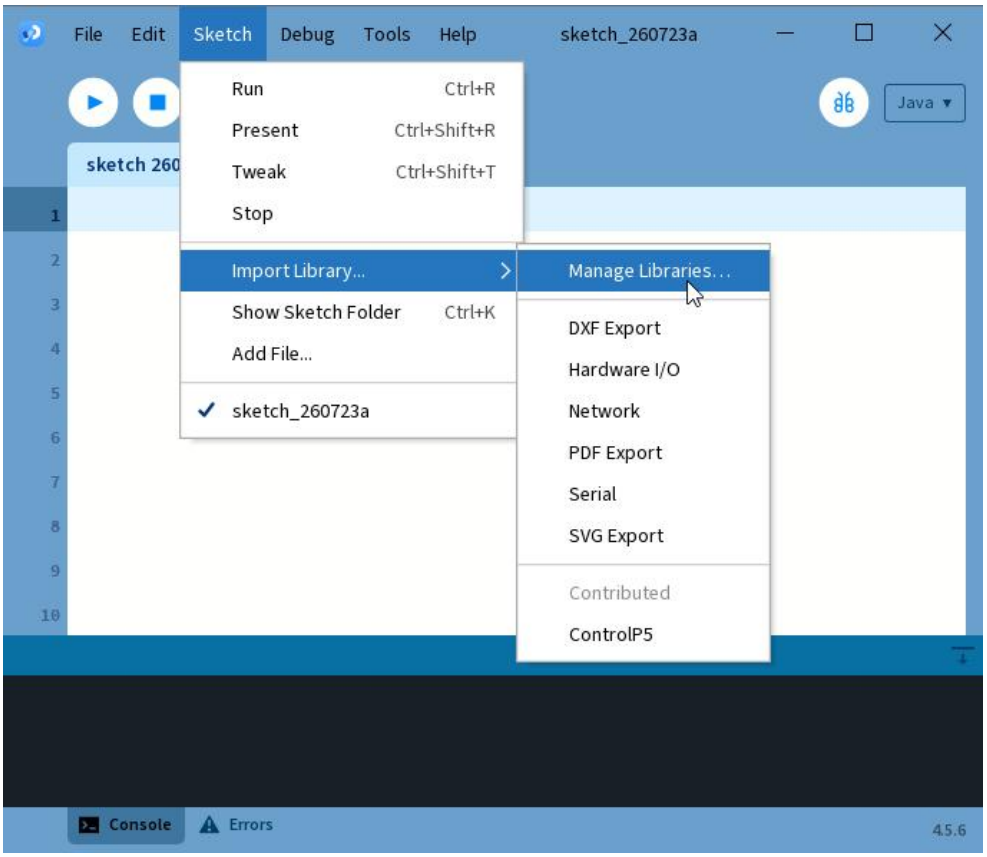


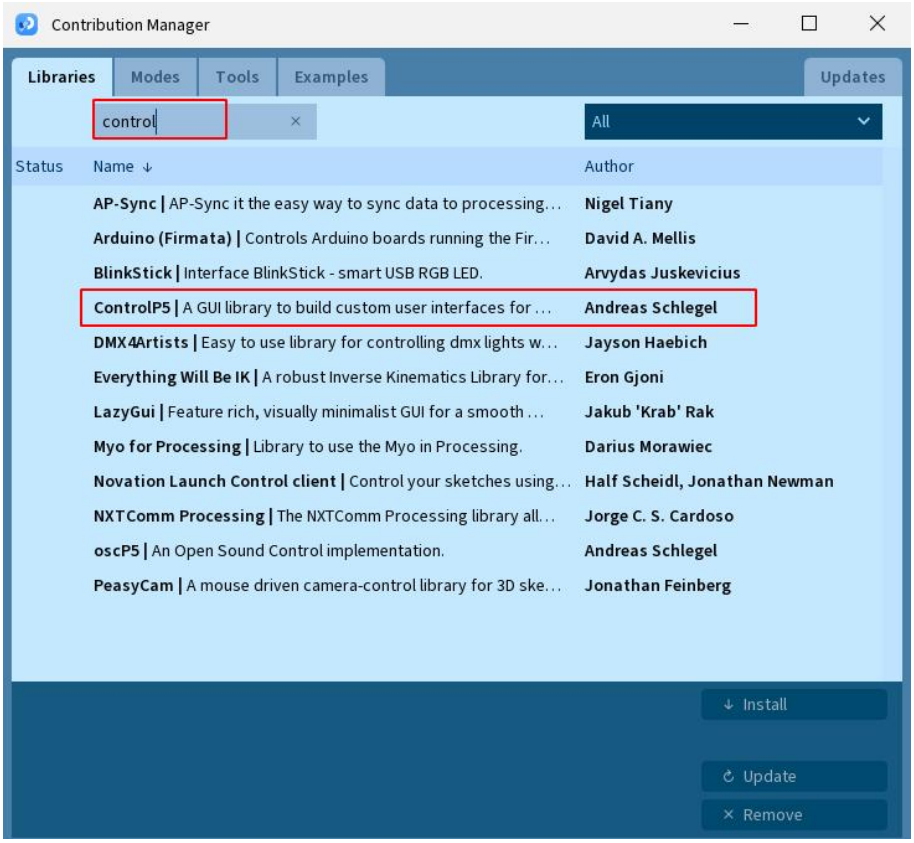


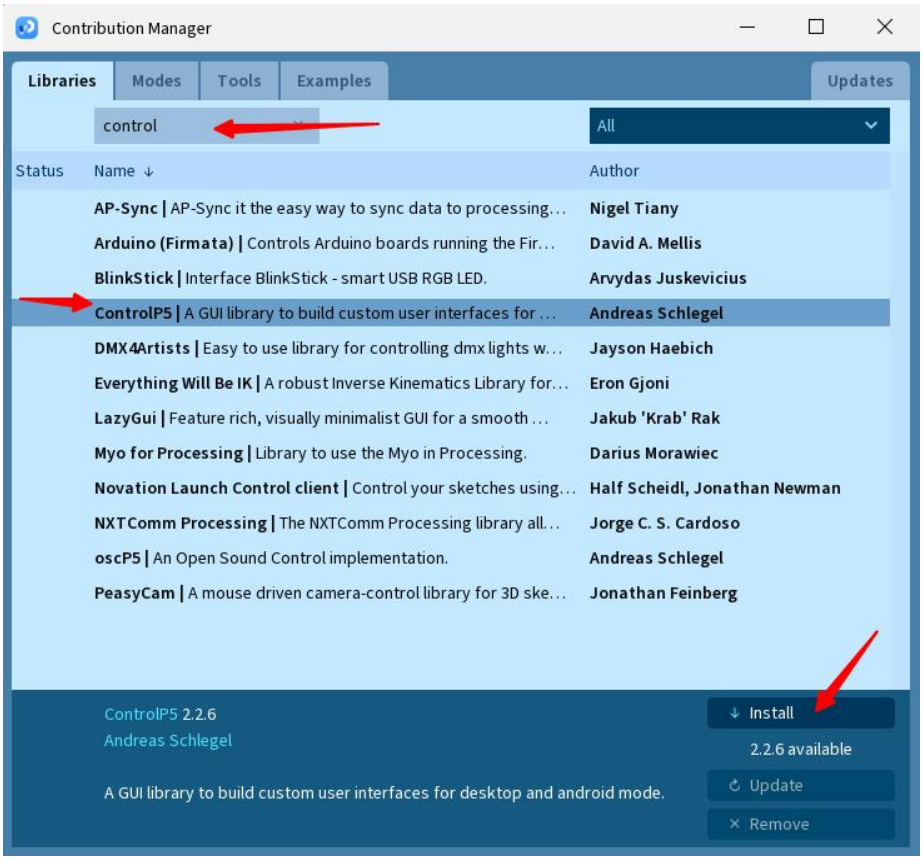
4.You have finished installing Processing. Open the software, select your preferred language, then restart Processing.

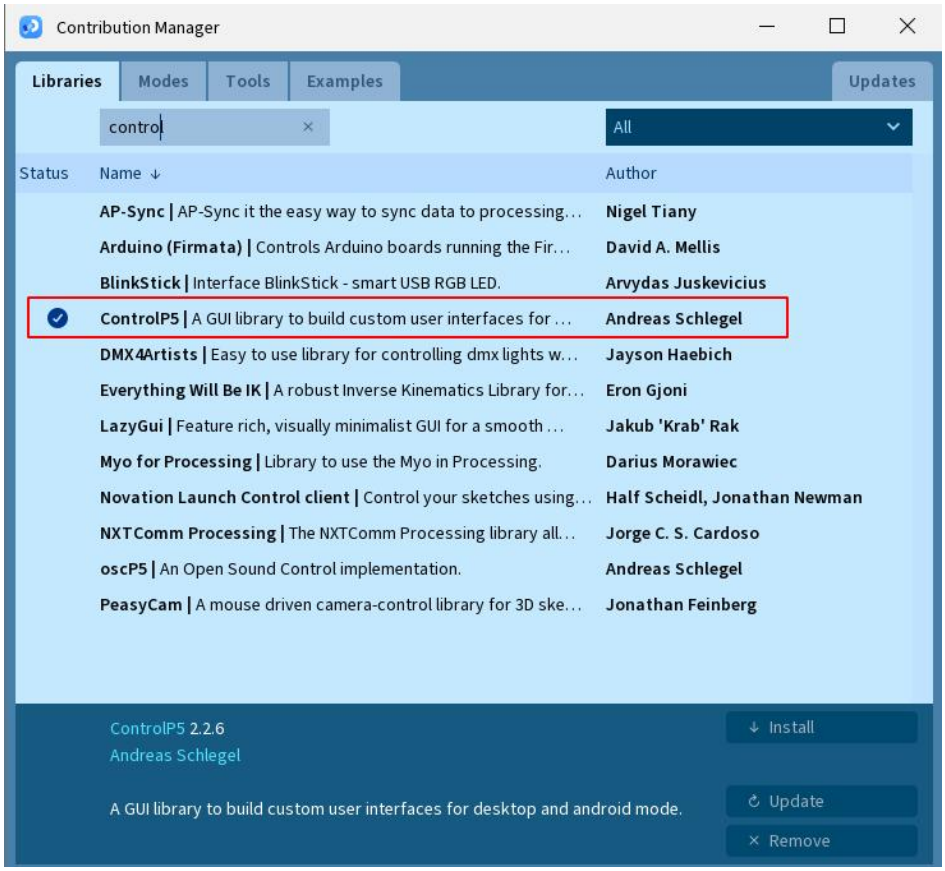


5.The ControlP5 library is used in our project, so we need to install it.




Contribution Manager
Libraries
Modes
Tools
Examples
Updates
control
All
Status
Name
Author
AP-Sync | AP-Sync it the easy way to sync data to processing...
Nigel Tiany
Arduino (Firmata) | Controls Arduino boards running the Fir...
David A. Mellis
BlinkStick | Interface BlinkStick - smart USB RGB LED.
Arvydas Juskevicius
ControlP5 | A GUI library to build custom user interfaces for ...
Andreas Schlegel
DMX4Artists | Easy to use library for controlling dmx lights w...
Jayson Haebich
Everything Will Be IK | A robust Inverse Kinematics Library for...
Eron Gjoni
LazyGui | Feature rich, visually minimalist GUI for a smooth ...
Jakub 'Krab' Rak
Myo for Processing | Library to use the Myo in Processing.
Darius Morawiec
Novation Launch Control client | Control your sketches using...
Half Scheidl, Jonathan Newman
NXTComm Processing | The NXTComm Processing library all...
Jorge C. S. Cardoso
oscP5 | An Open Sound Control implementation.
Andreas Schlegel
PeasyCam | A mouse driven camera-control library for 3D ske...
Jonathan Feinberg
Install
Update
Remove

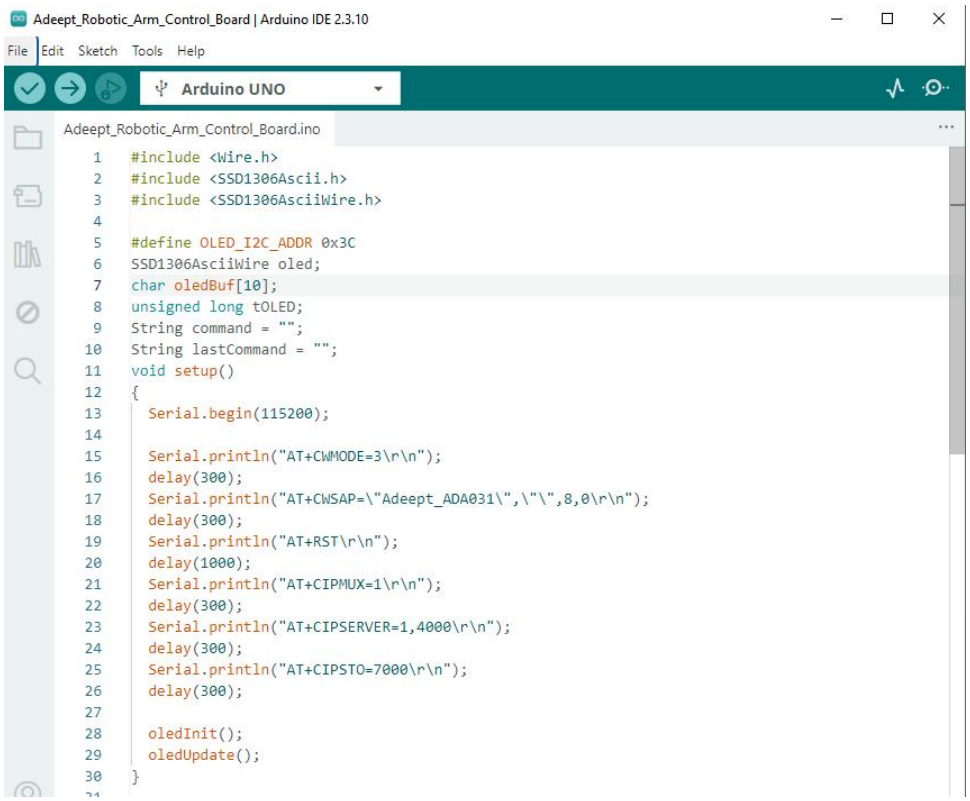

Contribution Manager
Libraries
Modes
Tools
Examples
Updates
control
All
Status
Name
Author
AP-Sync | AP-Sync it the easy way to sync data to processing...
Nigel Tiany
Arduino (Firmata) | Controls Arduino boards running the Fir...
David A. Mellis
BlinkStick | Interface BlinkStick - smart USB RGB LED.
Arvydas Juskevicius
ControlP5 | A GUI library to build custom user interfaces for ...
Andreas Schlegel
DMX4Artists | Easy to use library for controlling dmx lights w...
Jayson Haebich
Everything Will Be IK | A robust Inverse Kinematics Library for...
Eron Gjoni
LazyGui | Feature rich, visually minimalist GUI for a smooth ...
Jakub 'Krab' Rak
Myo for Processing | Library to use the Myo in Processing.
Darius Morawiec
Novation Launch Control client | Control your sketches using...
Half Scheidl, Jonathan Newman
NXTComm Processing | The NXTComm Processing library all...
Jorge C. S. Cardoso
oscP5 | An Open Sound Control implementation.
Andreas Schlegel
PeasyCam | A mouse driven camera-control library for 3D ske...
Jonathan Feinberg
ControlP5 2.2.6
Andreas Schlegel
A GUI library to build custom user interfaces for desktop and android mode.
Install
2.2.6 available
Update
Remove



11.6 Demonstration

1. Connect your computer and Adeept Robotic Arm Control Board (Arduino Board) with a USB cable.

2. Open "**11_ESP8266/Adeept_Robotic_Arm_Control_Board**" folder in "**/Code**", double-click "**Adeept_Robotic_Arm_Control_Board.ino**".



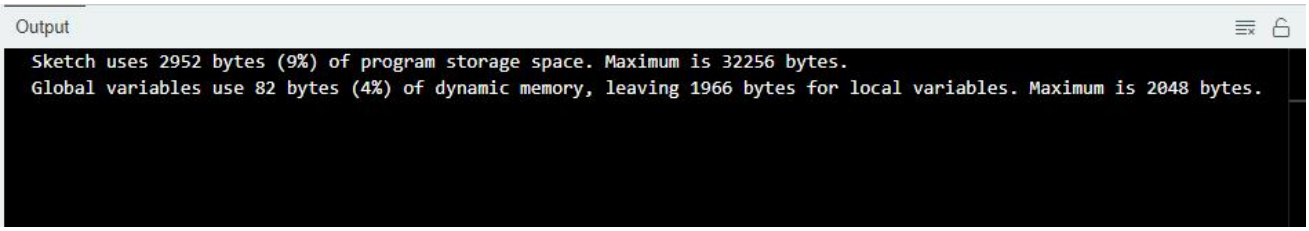
3. Select development board and serial port.

Board: **Tools--->Board--->Arduino AVR Boards--->Arduino Uno**

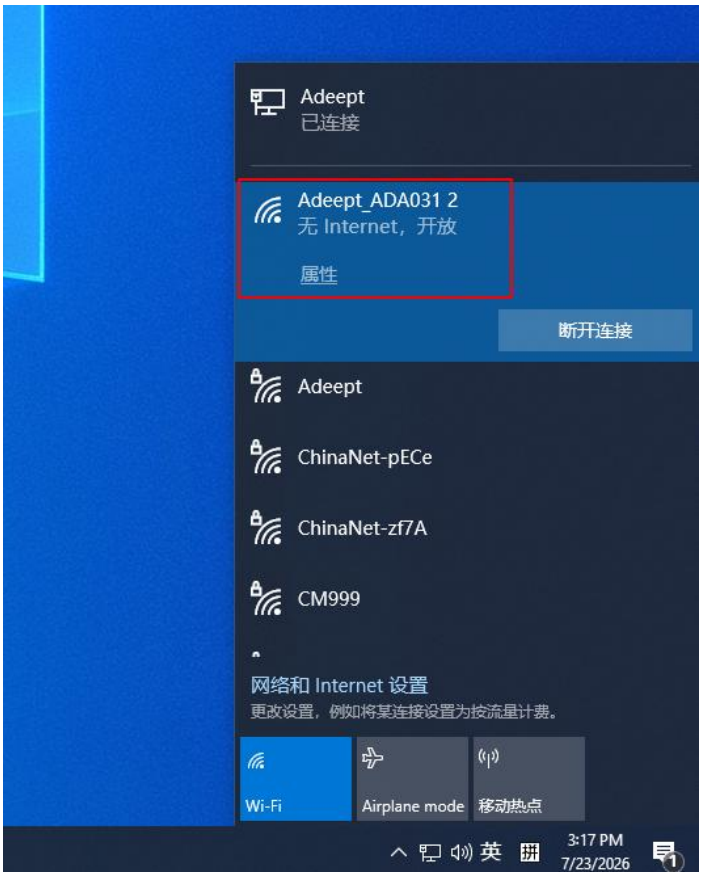
Port: **Tools --->Port--->COMx**

Note: The port number will be different in different computers.

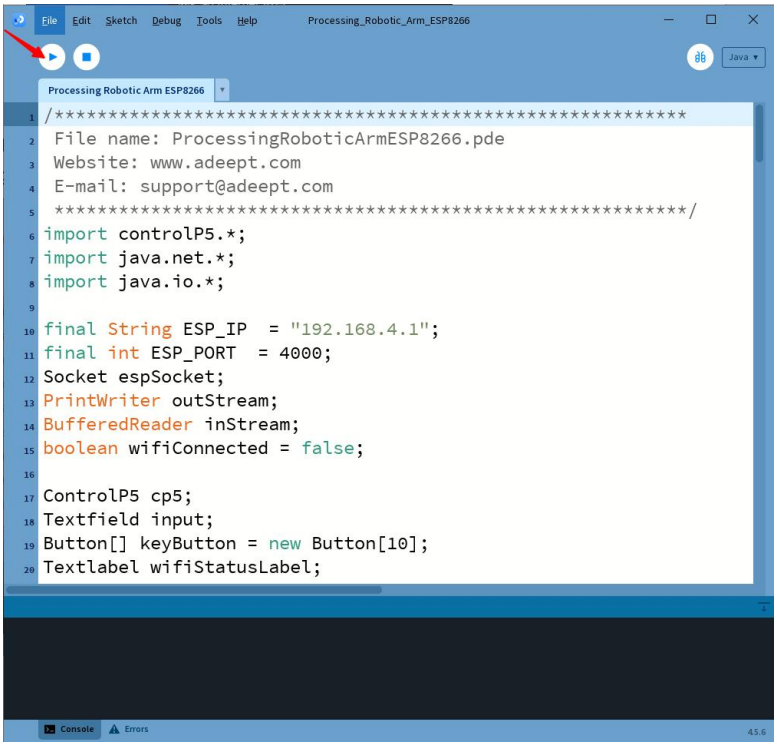
4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.



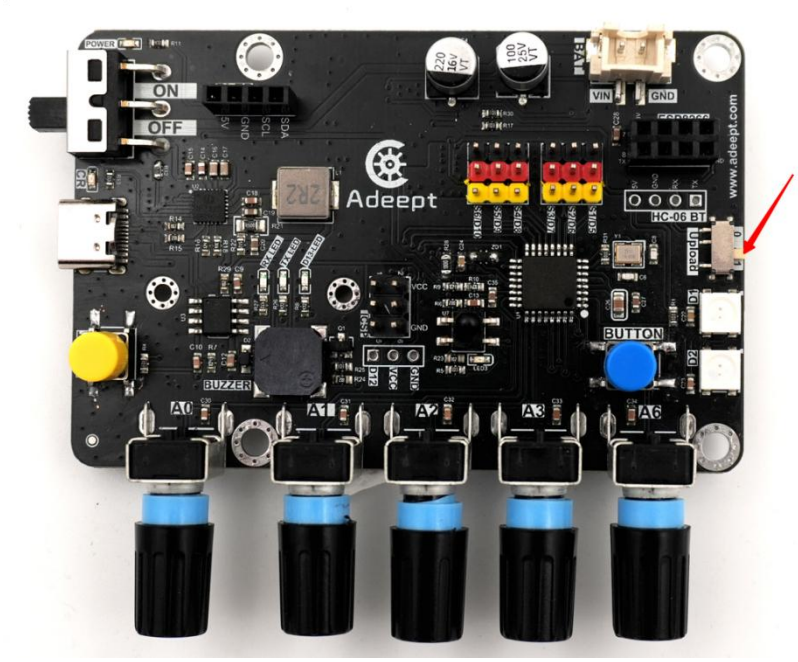
5. Once the program runs normally, the ESP8266 module will generate a password-free Wi-Fi hotspot named **Adeept_ADA031**, which we need to connect to via a computer.



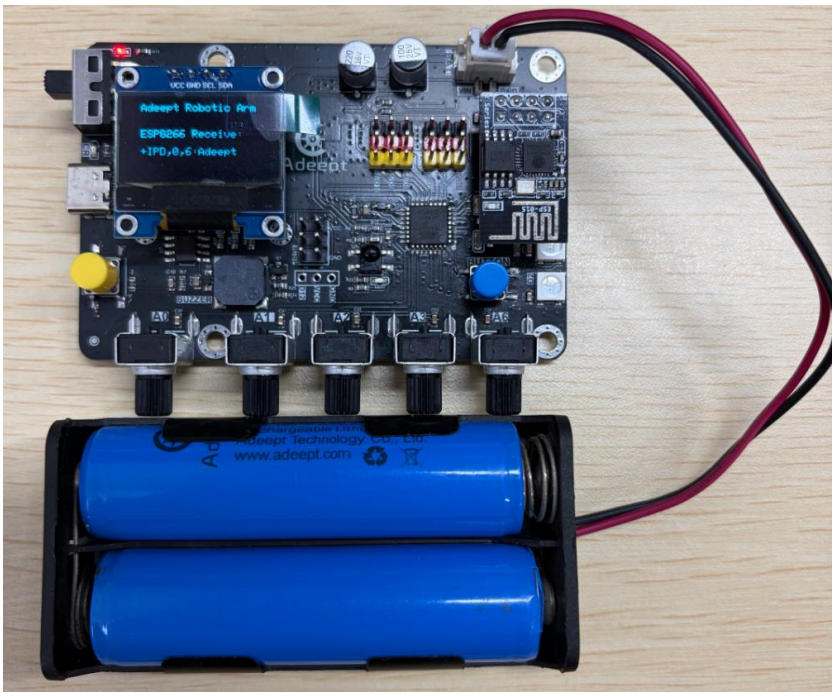
6. Open "**11_ESP8266/Processing_Robotic_Arm_ESP8266**" folder in "**/Code**", double-click "**Processing_Robotic_Arm_ESP8266.pde**".



7. After running the code, a graphical user interface pops up. The top-left corner indicates that the program has established a connection with the robot arm control board. However, we still need to toggle the upload switch on the control board to position **1** to switch the upload mode to the ESP8266 communication mode.



8. Now you can type text or press buttons on the operation interface, and the OLED screen will display the received data in real time.



9.If no data appears on the OLED after you send messages, troubleshoot the issue according to the following points:

a. Confirm that the correct firmware has been flashed to the control board.

b. Check that the ESP8266 module is properly inserted into the control board and the Upload switch on the control board is toggled to position 1.

c. Verify that your computer is connected to the Wi-Fi hotspot normally.

d. Restart the Processing program and re-establish the connection.

11.7 Code

```
#include <Wire.h>
#include <SSD1306Ascii.h>
#include <SSD1306AsciiWire.h>

#define OLED_I2C_ADDR 0x3C
SSD1306AsciiWire oled;
char oledBuf[10];
unsigned long tOLED;
String command = "";
String lastCommand = "";

void setup(){
  Serial.begin(115200);
```

```
14
15    Serial.println("AT+CWMODE=3\r\n");
16    delay(300);
17    Serial.println("AT+CWSAP=\"Adeept_ADA031\",\"\",8,0\r\n");
18    delay(300);
19    Serial.println("AT+RST\r\n");
20    delay(1000);
21    Serial.println("AT+CIPMUX=1\r\n");
22    delay(300);
23    Serial.println("AT+CIPSERVER=1,4000\r\n");
24    delay(300);
25    Serial.println("AT+CIPSTO=7000\r\n");
26    delay(300);
27
28    oledInit();
29    oledUpdate();
30  }
31
32  void loop(){
33    while(Serial.available()){
34      command = Serial.readStringUntil('\n');
35    }
36    oledUpdate();
37    delay(10);
38  }
39
40  void oledInit(){
41    Wire.begin();
42    oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
43    oled.setFont(Adafruit5x7);
44    oled.clear();
45    oled.set1X();
46
47    oled.setCursor(10,0);
48    oled.print("Adeept Robotic Arm");
49
50    oled.setCursor(10,3);
51    oled.print("ESP8266 Receive:");
52    oled.setCursor(10,5);
53    oled.print(command);
54  }
55
56
57  void oledUpdate(){
58    if(millis()-tOLED<100)return;
59    tOLED=millis();
60    if(lastCommand != command){
61      oled.setCursor(10,5);
62      oled.print("                      ");
63      oled.setCursor(10,5);
64      oled.print(command);
65      lastCommand = command;
66    }
67  }
```